

COS214 Practical 6

✨ **TIVIDY: Workflow Management System** ✨

- Jayden Smith - u24583741
- Declan Mc Cullough -u21760022
- Yash Ghela -u25161360
- Rei Piater-Boswell -u25678592
- Leanne van der Horst - u24566935

Task1: Research Workflow Management Systems

When designing our workflow management system we found that a workflow is defined as a system for managing repetitive processes and tasks which occur in a particular order, a workflow management system defines, manages and executes workflows. A workflow definition is static while a running workflow is the actual running instance of the workflow. In most systems, work items and activities are separated as static definitions vs active processes, this allows for them to be managed differently. When assigning items we found it was best to use a role based map that allows for an item to be assigned to the relevant participant as fast as possible, if they cannot do it then that item needs to be moved to the closest next person who can handle the item. Generally, participants are registered with personal details, a role such as intern, manager etc and then a system defined responsibility level that can be used when mapping roles. We have built our approval, rejection and escalation system around a reviewer role that will be given to a participant rather than an automated system as our scenario will call for a more detailed look from a person rather than system. We have modelled our system such that certain items will depend on other work items to be completed before they can begin, allowing for a sequential and parallel workflow system. When events occur the relevant states are changed to pass data when a message is received, if that event fails we use versioning to rollback to the last stable state and preserve data. Pre built APIs or connectors are generally the best way to interact with external systems. In order to maintain a history, snapshots of a system are taken either in real time or in a pre defined gap. This creates a responsive roll back system. Running workflows are monitored using analytics systems that allow you to see who is working where and on what tasks, as well as computing the amount of changes occurring. A workflow engine is reusable across multiple different processes, because it generalises how work is done, not what work is done, a restaurant can use a workflow system and a movie studio can use the same system if their workflows require the same elements. It decouples the process from a blueprint. An important aspect of a system is to make nested activities directly linked to their parent so they can easily move and change states within while acting as dependencies. For workflow systems to work for different processes, being able to configure internal business rules is important. When work is in progress but we need to change workflow definitions, we have to send notifications out to the relevant work items including cancellations, delays or changes where relevant. All these details allow for a powerful workflow system that can handle and easily adjust to business needs.

References:

<https://www.ibm.com/think/topics/workflow>

<https://www.sciencedirect.com/topics/computer-science/workflow-management-system>

<https://www.ibm.com/docs/en/openpages/9.0.0?topic=fundamentals-workflow-definitions-workflow-instances>

<https://www.atlassian.com/work-management/project-management/approval-process-workflow>

<https://www.ibm.com/think/topics/workflow-engine>

Task2: Define the Scenario and System

Initial scope of TIVIDY

TIVIDY is a generic workflow management system that defines, manages and executes workflows. It generalises how work is done rather than what work is done, so the same engine could serve a restaurant or an animation studio. Its main responsibilities are creating running instances from static definitions, managing work-item lifecycles and dependencies. It also reacts to events, integrates with external systems through connectors, keeps snapshots and an audit history for rollback, and provides monitoring and analytics. A definition holds the static structure, dependencies, roles and rules, while each instance holds its own state. TIVIDY coordinates work but does not perform it, and we expect assignment rules, approval structure, work-item behaviour, integrations and reporting to vary independently.

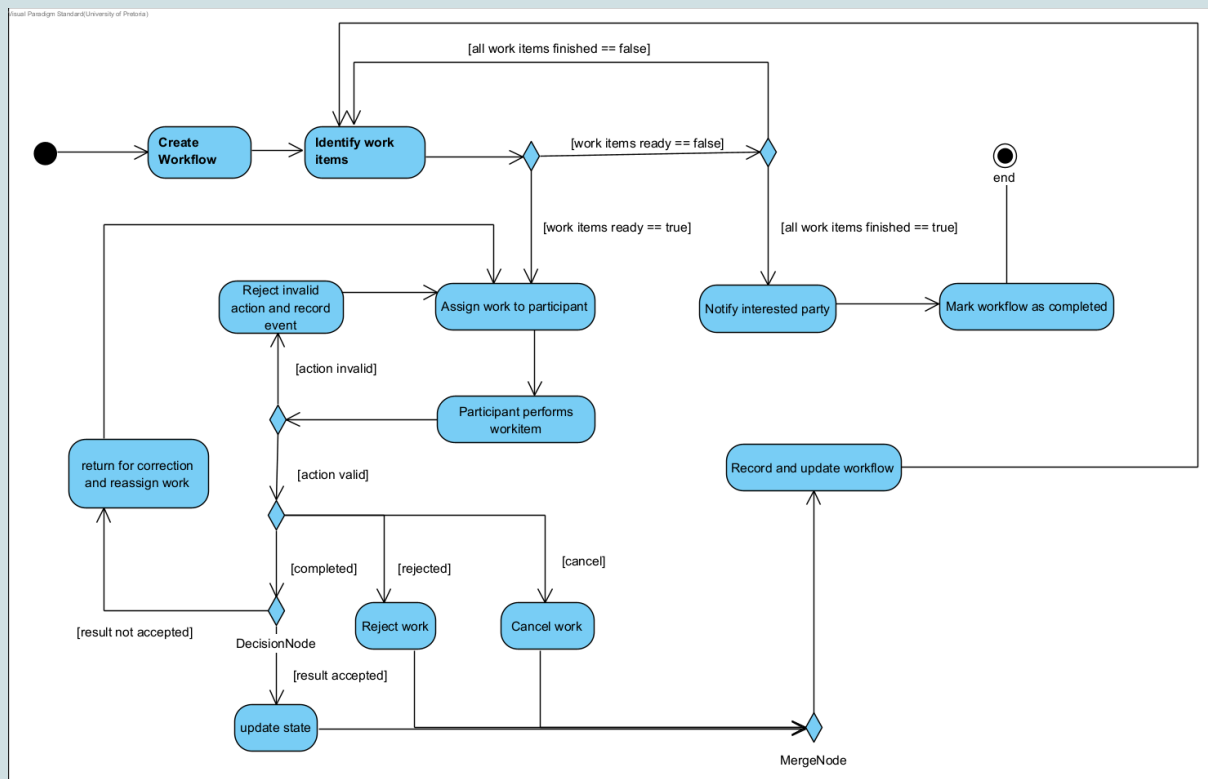
- **Organization and environment:** Our animation studio is a mid-sized studio producing animated shorts for external clients. About 15 people work across departments, and many tasks cannot start until others finish. Handovers and approvals are currently tracked through chat and spreadsheets, which causes lost approvals and unclear versions.
- **Process being managed:** The production of the short from client brief to final delivery. The main workflow contains one nested sub-process for each shot.
- **Participants and roles:** Interns and junior artists handle simple tasks, and artists such as animators and compositors do the core production work. Pipeline TDs handle technical failures, the Art Director and Director act as reviewers, and the Producer handles escalations and scope changes. The client is an external reviewer at defined milestones.
- **Major pieces of work:** Pre-production covers the script, storyboard, animatic and designs. Each shot then goes through layout, animation, rendering and compositing, followed by post-production (sound, edit, colour grade) and delivery.
- **Decisions and routing points:** Each task goes to the participant with the matching role and lowest current load, and moves to the next higher responsibility level if they cannot take it. Every review ends in approved, revisions needed or rejected, with escalation to the Producer after a set number of revision rounds.
- **Approvals and dependencies:** The animatic needs Director and then Client approval before production begins, and character designs need Art Director approval before modelling. Animation needs an approved layout, and compositing needs all render passes. Shots can run in parallel, but the final edit needs every shot composited.
- **Failures and exceptions:** A reviewer who does not respond in time causes the review to escalate. A failed render is retried and then escalated to the Pipeline TD, and a failed event or client rejection rolls the workflow back to its last stable or approved state. If a shot is cut, its work items are cancelled and dependants are notified.
- **External systems:** A render farm receives jobs and returns pass or fail results. Asset management stores files and versions, a client review portal handles client approvals, and a notification service sends emails or chat messages. An invoicing system is used at delivery.

- **Expected outcome:** The approved short is delivered to the client. Every assignment, approval, revision, failure and rollback is recorded, and reports show who worked on what and where time was lost.

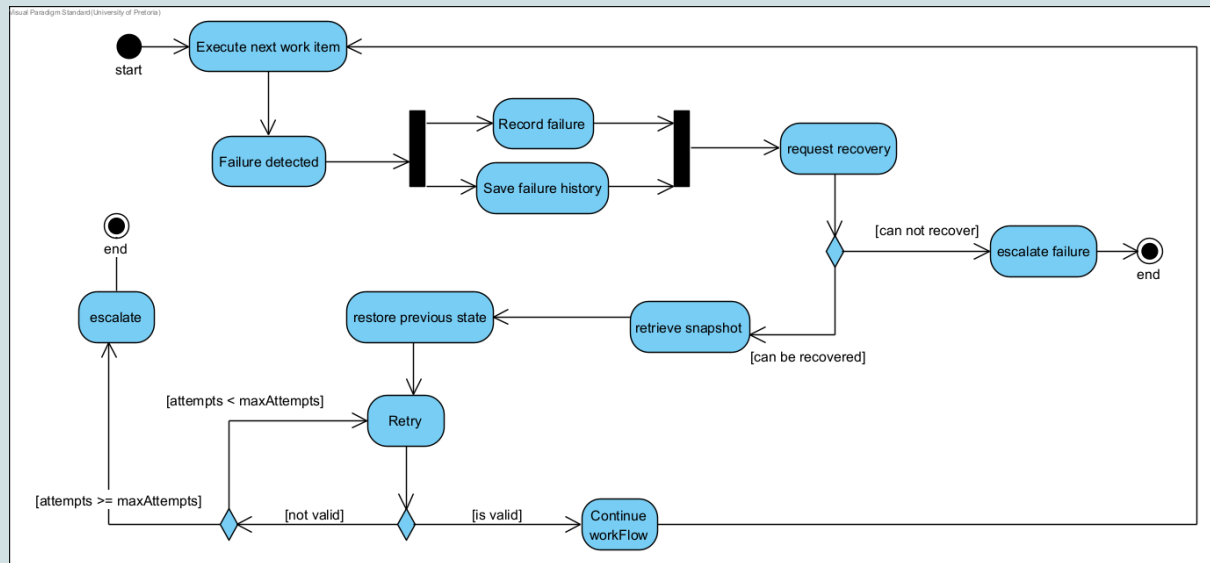
Task3: UML activity diagrams

Activity diagram 1 - general workflow execution

(Made some assumptions on the system since task2 has not been fully defined. I'm not sure this is correct but _(ツ)_/)

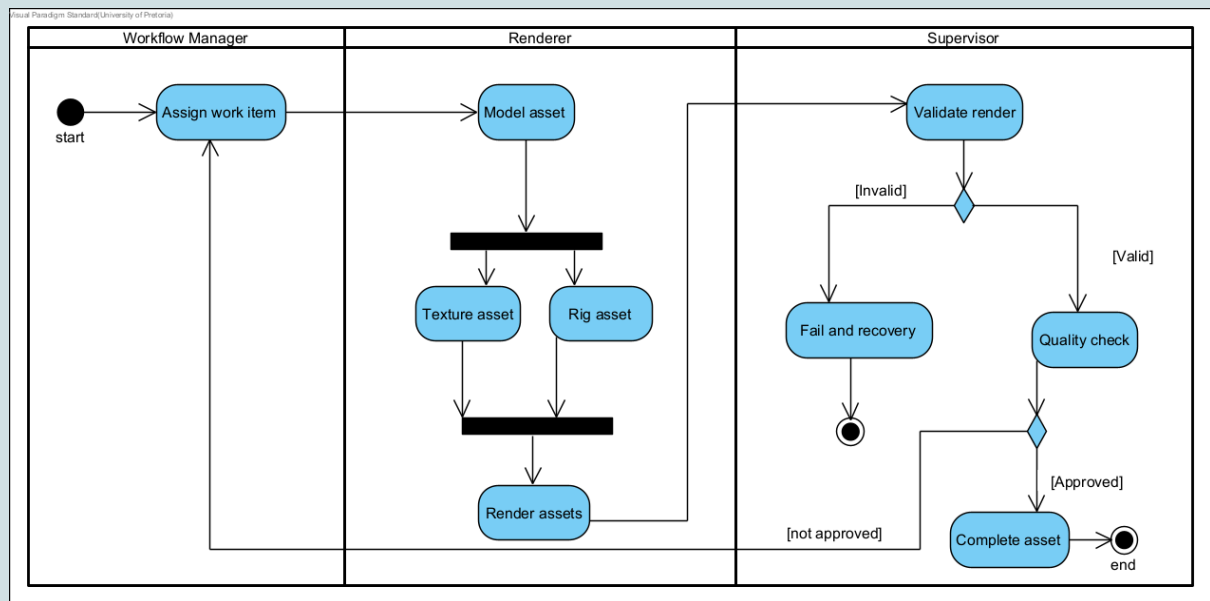


Activity diagram 2 - general workflow execution. Failure and recovery.



Activity diagram 3 - scenario diagram

Workflow of manager assigning work item to Renderer and work item being executed and validated by supervisor



Task4:

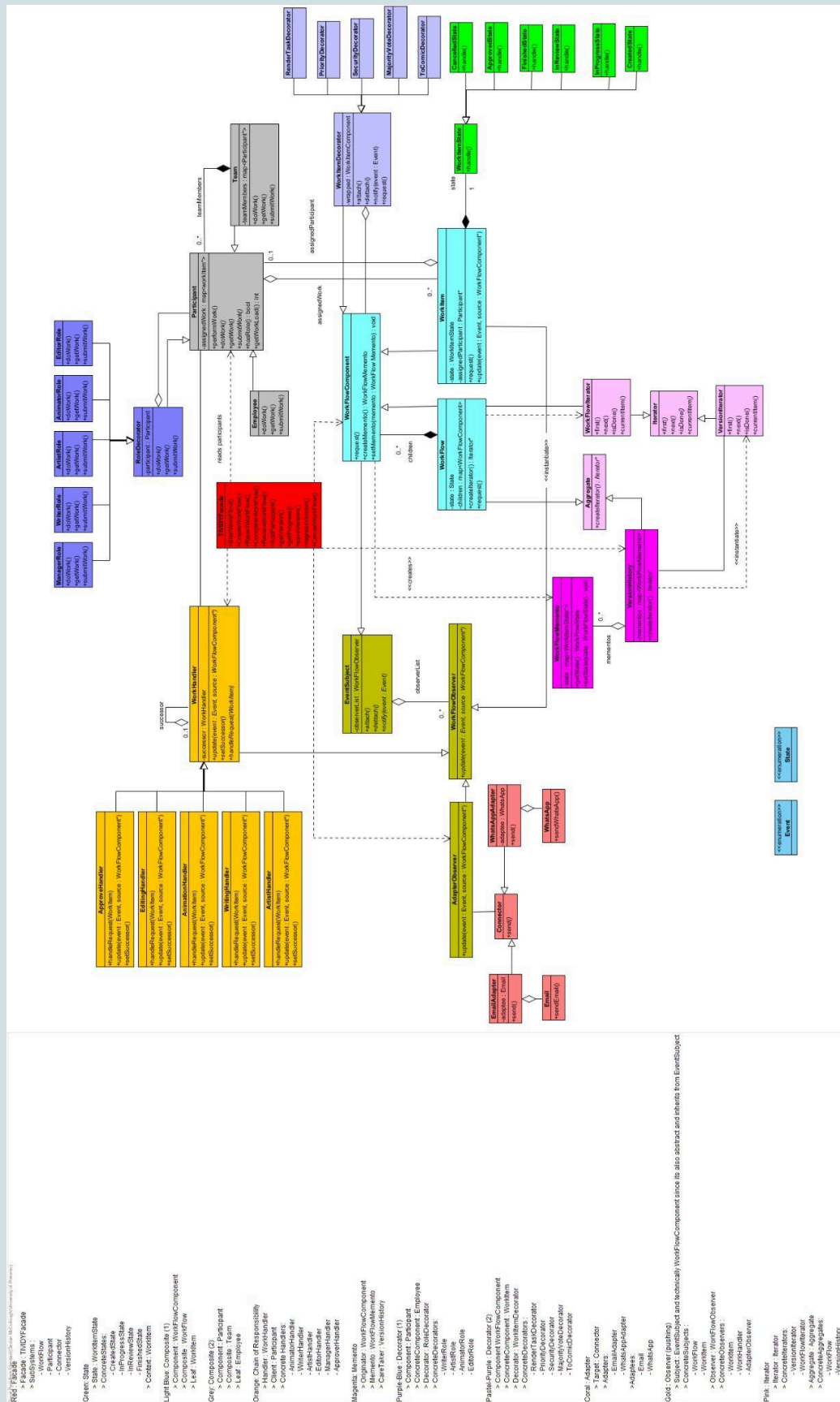
Pattern	Use
Composite Participants 1: • Component : ◦ WorkflowComponent • Composite : ◦ Workflow • Leaf : ◦ WorkItem • Client : ◦ TIVIDYFacade Participants 2: • Component : ◦ Participant • Composite : ◦ Team • Leaf : ◦ Employee • Client : ◦ TIVIDYFacade	Composite 1 • solves the need to treat multiple work items uniformly and have them be in a part-whole hierarchy within work flows. • WorkflowComponent provides an interface for TIVIDYFacade; Workflow contains WorkItems and potentially other WorkFlows. • The part that can potentially change are the functions within the pattern and the use of Workflow containing other Workflow objects. Composite 2 • solves the need to treat multiple employees uniformly and have them be in a part-whole hierarchy within teams. • Participant provides an interface for TIVIDYFacade; Team contains Employees and potentially other Teams. • The part that can potentially change are the functions within the pattern and the use of Teams containing other Team objects.
Decorator Participants 1: • Component: ◦ WorkflowComponent • ConcreteComponent : ◦ WorkItem • Decorator : ◦ WorkItemDecorator • ConcreteDecorators: ◦ PriorityDecorator ◦ SecurityDecorator ◦ ToComicDecorator ◦ RenderTaskDecorator Participants 2: • Component: ◦ Participant • ConcreteComponent: ◦ Employee • Decorator : ◦ RoleDecorator • ConcreteDecorators:	Decorator 1 • The Decorator design pattern allows the attachment of additional responsibilities to WorkItems at run time. This will allow for the use of a generic WorkItem that can be added onto to make it more unique and adapted to specified use. • The WorkflowComponent will act as an interface for WorkItems that can have ConcreteDecorators added to them; The WorkItemDecorator will hold a reference to a WorkItem object. The ConcreteDecorators will then add stackable responsibilities to the Work Item object such as turning it into a comic instead of an animation or locking it away if its unfinished work. • The part that can potentially change are the functions within the pattern as well as the actual types of ConcreteDecorators. Decorator 2 • The Decorator design pattern allows the attachment of additional responsibilities to Employees at run time. This will allow for the use of a generic Employee that can be added onto to make it more unique and adapted to specified

○ WriterRole ○ ArtistRole ○ AnimatorRole ○ EditorRole ○ ManagerRole	roles in the company and animation pipeline. • The Participant will act as an interface for Employees that can have ConcreteDecorators added to them; The RoleDecorator will hold a reference to an Employee object. The ConcreteDecorators will then add stackable responsibilities to the Employee object such as being able to take on the editing stretch of a WorkItem or Writing a sequence to be animated. • The part that can potentially change are the functions within the pattern as well as the actual types of ConcreteDecorators.
Facade Participants • Facade : ○ TIVIDYFacade • SubSystems : ○ WorkFlow ○ Participant ○ Connector ○ VersionHistory	• TIVIDYFacade will allow a user to interact with the whole system at a simplified level so they do not have to know all the ins and outs of every pattern. It helps connect patterns to each other too. • Within the facade the user will have the ability to begin, review, reject, cancel or complete a workflow; add a participant to a workItem; get progress of a workflow; get, save or regress a version of a workItem • The parts that can potentially change are the functions within the pattern and what it can do since some parts a manager should only be able to do.
Chain of Responsibility Participants • Handler : ○ WorkHandler • Client : ○ Participant • ConcreteHandlers : ○ AnimatingHandler ○ WritingHandler ○ ArtistHandler ○ EditingHandler ○ ApproveHandler	• Gives multiple participants the chance to handle the workItem that is in progress, while decoupling the decorators of the participants. Chains the employee objects and sends the workItem to the next employee with that role if they have less of a workload. • WorkHandler defines the interface for handling the requests and implements the chaining process by linking the next employee to the chain; Participant will initiate the request to be sent down the chain when a workItem enters the State corresponding to a ConcreteHandler; The ConcreteHandlers will handle the request of the WorkItem object being passed along the chain until it reaches an employee with a lower workload. • The parts that can potentially change are the functions within the pattern and what ConcreteHandlers we have since the system may have to change.
Adapter Participants • Target : ○ Connector	• The adapter will adapt notifications in order to send them to employees on WhatsApp and/or Email. • The AdapterObserver will wait for a change in the WorkItem and when this takes place it will call the

• Adapter : ◦ EmailAdapter ◦ WhatsAppAdapter • Adaptee : ◦ Email ◦ WhatsApp • Client ◦ AdapterObserver	request function of the Connector which will intern delegate the request to the two Adapters and they will send the relevant information as an object to the Adaptees which then send the notifications. • The parts that can potentially change are the functions within the pattern and what it can do since we may have to use the Adapter in a different external system.
Iterator Participants • Iterator : ◦ Iterator • ConcreteIterator : ◦ VersionIterator ◦ WorkflowIterator • Aggregate : ◦ Aggregate • ConcreteAggregate : ◦ Workflow ◦ VersionHistory	• The Iterator will be able to go through the composite of work items to check the number of WorkItems as well as iterate through the memento map to count version numbers, all to get statistics of WorkFlows. • The Iterator defines the interface for accessing and traversing through the Workflow and the VersionHistory; the ConcreteIterators implement the Iterator and go through these ConcreteAggregates by using the Aggregate as an interface. • We believe there will be little to no changes in this pattern, except adding a new ConcreteIterator to traverse other patterns like counting the number of current employees.
Memento Participants • Originator : ◦ WorkflowComponent • Memento : ◦ WorkflowMemento • CareTaker : ◦ VersionHistory	• The Memento will be able to hold revisions of workItems and WorkFlows and will allow restoration and tracking of their states. • The WorkflowComponent holds a state at all times, and creates a snapshot of this state and this gets stored in the WorkflowMemento and the VersionHistory keeps the WorkflowMemento safe. • The use of what states are stored in the snapshot might change.
Observer Participants • Subject : ◦ EventSubject • ConcreteSubject: ◦ Workflow ◦ WorkItem • Observer : ◦ WorkflowObserver • ConcreteObserver : ◦ WorkItem ◦ WorkHandler ◦ AdapterObserver	• This Pattern solves the problem of multiple other patterns having to know and change according to the WorkflowComponents automatically. The Adapter, the WorkItems, and the Handlers. • The EventSubject defines an interface for observers to attach and detach to the WorkFlows and the WorkItems. The WorkflowObserver defines the interface for the ConcreteObservers which maintain a reference to the subjects they are observing and get notified when they change • The number of classes that observe the WorkFlows and WorkItems may need to change and the Subjects may need to include other object types
State	• The State Pattern allows the WorkItems to have

Participants • State : ◦ WorkItemState • ConcreteStates : ◦ CreatedState ◦ InProgressState ◦ InReviewState ◦ FinishedState ◦ CancelledState • Context : ◦ WorkItem	differing states that invoke different changes according to what that state currently is. This will allow for dynamic behaviour at runtime. • The WorkItemState defines the interface for encapsulating behaviour associated with a particular state of the WorkItem. The ConcreteStates implement the different behaviour associated with a state of a WorkItem • The types of ConcreteStates may change as the system gets developed.
Template Method Participants • AbstractClass : ◦ Participant • ConcreteClass : ◦ Employee ◦ Team • templateMethod : ◦ performWork • primitiveOperation : ◦ doWork ◦ getWork ◦ submitWork	• The Template method allows one function to be called in the Participant Composite and depending on the decorator, the primitive operations do different actions. • The Participant implements performWork as the template method and defines abstract primitive operations, doWork, getWork, submitWork, which then get implemented in the ConcreteClasses. • We do not foresee a change in the Template Method unless it gets changed for a different design pattern.

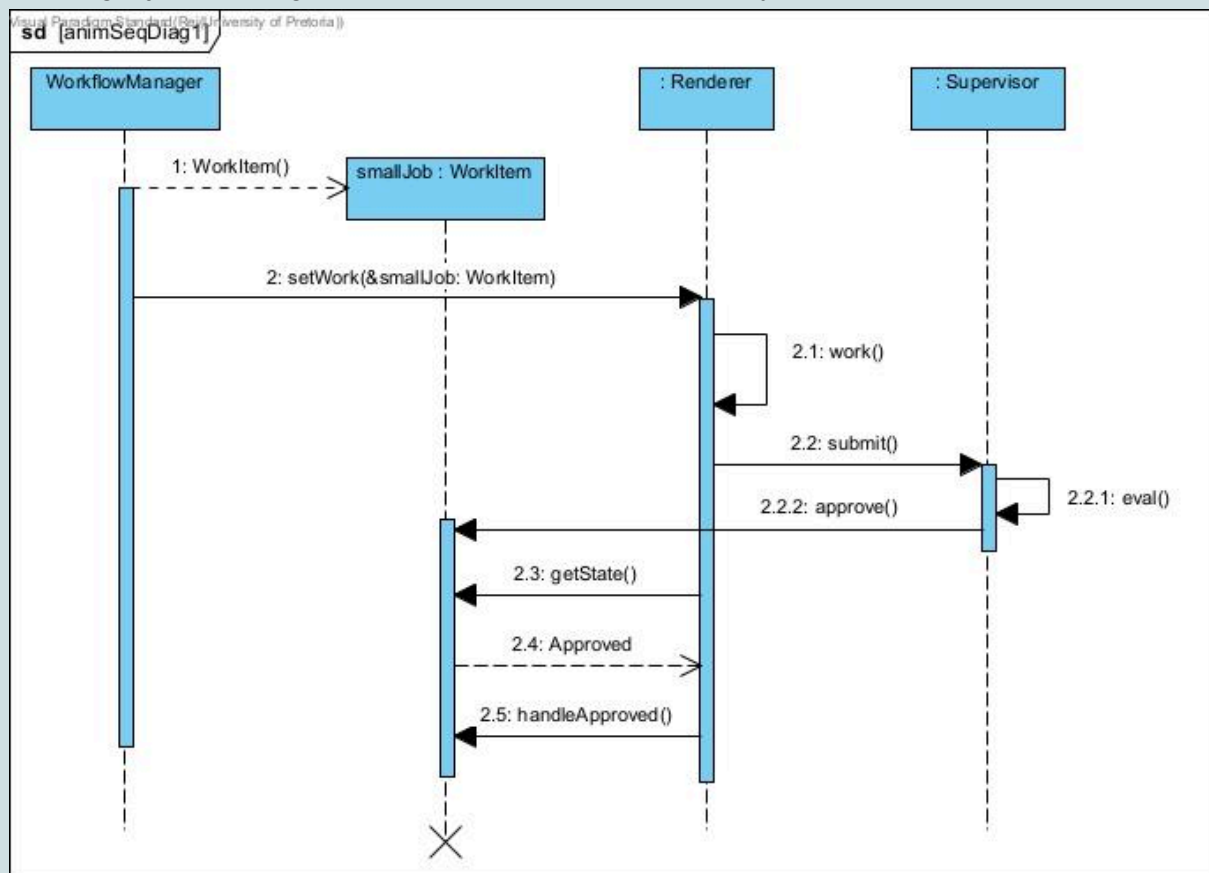
Task5:



Task6:

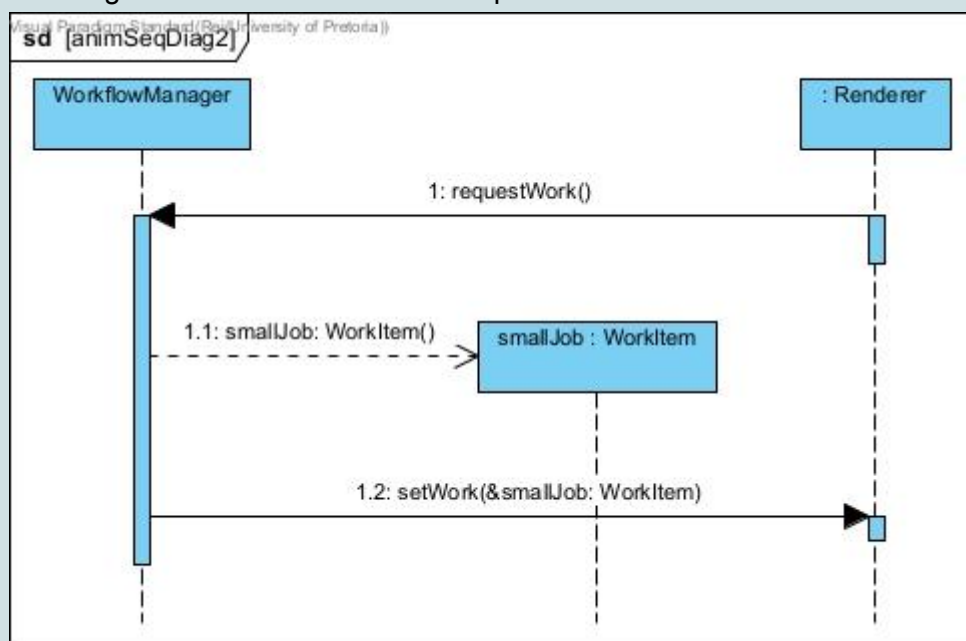
Sequence Diagram 1:

Creating a job, working on it and handle what reviewer "says"



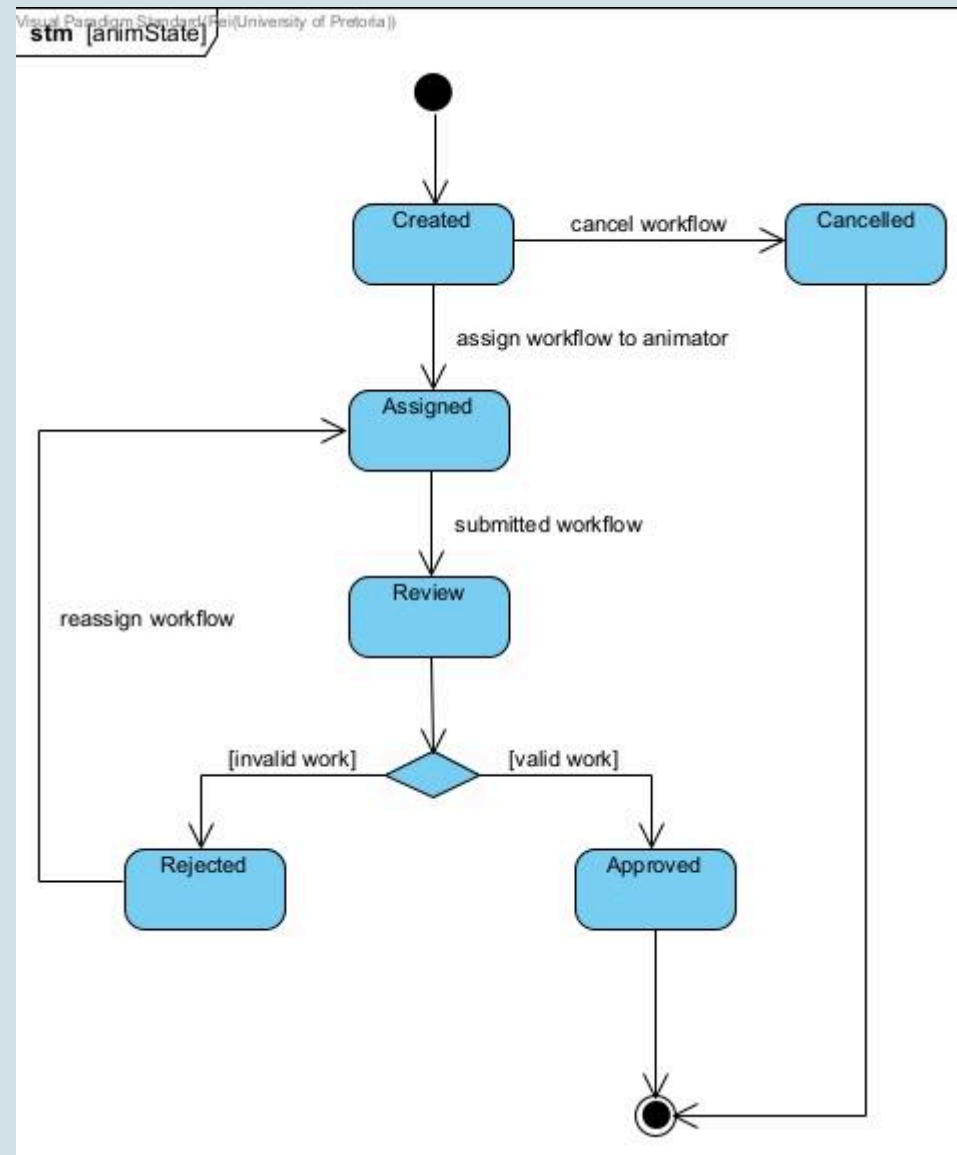
Sequence Diagram 2:

Handling an event where a worker requests to have a WorkItem



State Diagram:

State of WorkItem



Task7:

- We adjusted participants to have teams as their composite
- Changed the role of participant admin to reviewer
- References added

Task8:

Github: <https://github.com/yashGhela/cos214prac6.git>